

Documentação Técnica — Padronização dos Códigos de Serviço da Gerdau

Visão geral

Este documento especifica a solução integrada para padronizar, pesquisar, cadastrar e governar códigos de serviço utilizados pela Gerdau. A proposta combina um módulo de dados e inteligência de busca com uma aplicação desktop em Java, criando um fluxo que:

1. saneia a base histórica;
2. separa os componentes de um código de serviço;
3. facilita a localização de itens existentes;
4. reduz a criação de duplicidades;
5. mantém rastreabilidade e controle;
6. prepara os dados para consolidação de compras e análise de gastos.

A solução foi desenhada para atender especialmente às categorias-piloto de Eletromecânica, Facilities e Civil, mas o modelo pode ser expandido para outras categorias.

1. Problema de negócio

1.1 Causa-raiz

O cadastro de contratos e serviços no SAP é realizado de forma descentralizada. Compradores ou fornecedores podem criar descrições sem um padrão único, sem validação de similaridade e sem aprovação prévia.

Esse cenário permite que um mesmo serviço seja registrado várias vezes com abreviações, ordens de palavras, códigos internos e descrições diferentes. Exemplos conceituais:

- SV-MEC-01-HN
- MEC.MANUT.NORM

- COD 4521 - MEC

Mesmo que representem a mesma atividade, o sistema pode interpretar cada registro como um item diferente.

1.2 Impactos

- duplicação e fragmentação do catálogo;
- dificuldade para localizar o código correto;
- comparação inconsistente de tarifas entre fornecedores;
- perda de poder de negociação por volume;
- dificuldade para consolidar o spend e realizar Spend Analysis;
- manutenção de códigos esquecidos ou sem uso;
- aumento do trabalho manual do time de compras;
- risco de decisões baseadas em dados incompletos.

1.3 Restrição do SAP

O SAP limita a descrição a até 40 caracteres. Essa limitação não é, por si só, o problema: o problema é utilizar os caracteres sem uma convenção de escrita. A solução deve, portanto, validar esse limite antes do envio e orientar o usuário a escrever uma descrição curta, consistente e suficientemente informativa.

2. Objetivos da solução

2.1 Objetivo principal

Criar uma camada de padronização e controle para os códigos de serviço, permitindo que a organização reutilize itens existentes e controle a criação de novos itens.

2.2 Objetivos específicos

- identificar e consolidar duplicidades históricas;
- decompor o código completo em quatro componentes reutilizáveis;
- permitir busca por ID, nome ou descrição;
- aplicar pesquisa aproximada para erros de digitação e variações textuais;

- validar o limite de 40 caracteres;
- bloquear ou encaminhar novos cadastros semelhantes para revisão;
- registrar usuário, data, hora e composição do código criado;
- preservar o relacionamento entre códigos antigos e padronizados;
- preparar uma base que possa ser integrada ao SAP ou a outros sistemas corporativos.

3. Princípios de arquitetura

A arquitetura segue os seguintes princípios:

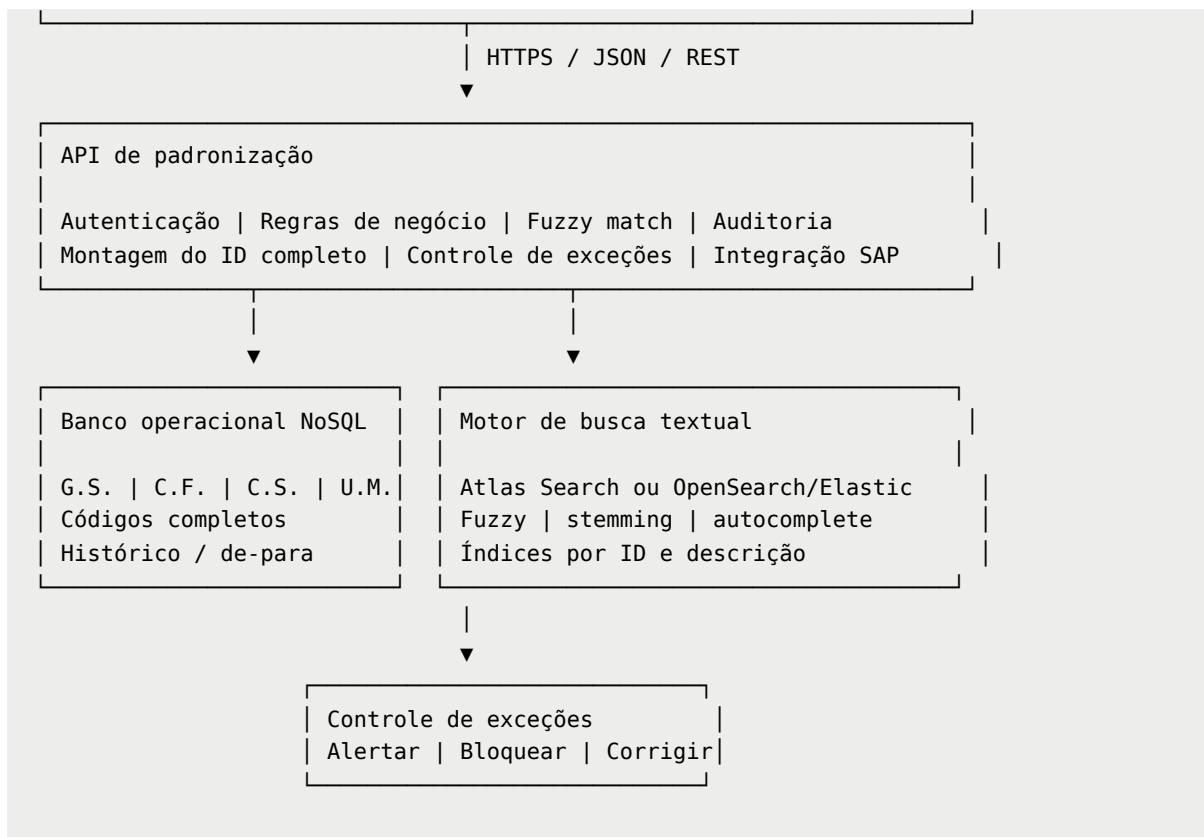
- **Reutilização antes de criação:** o sistema deve tentar encontrar um item equivalente antes de aceitar um novo cadastro.
- **Separação de responsabilidades:** a interface cuida da experiência do usuário; a API aplica regras; o banco armazena e pesquisa dados.
- **Controle por exceção:** casos claramente novos podem seguir o fluxo normal; casos similares recebem alerta ou bloqueio conforme a similaridade.
- **Rastreabilidade:** toda criação, alteração ou rejeição deve ser identificável.
- **Compatibilidade com legado:** dados antigos devem ser importados mesmo quando não possuem usuário ou timestamp.
- **Busca orientada a texto:** o usuário não deve precisar conhecer exatamente o ID para encontrar um item.
- **Validação no servidor:** regras importantes não podem existir apenas na interface.
- **Evolução incremental:** a primeira versão deve ser simples e permitir trocar o mecanismo de busca sem reescrever toda a aplicação.

4. Arquitetura integrada

Aplicação desktop JavaFX

Seletores de G.S. | C.F. | C.S. | U.M.

Cadastro de item | Validação | Feedback de similaridade



4.1 Camadas

Camada de apresentação

Aplicação desktop desenvolvida em JavaFX. Exibe os seletores, os resultados de busca, os alertas de duplicidade e os estados do cadastro.

Camada de aplicação

API responsável por autenticação, validações, montagem de códigos, busca, auditoria e integração com o banco. Essa camada evita que a aplicação desktop acesse o banco diretamente.

Camada de dados

Banco NoSQL para documentos e entidades do catálogo, combinado com um mecanismo de busca textual. O banco guarda o estado oficial; o mecanismo de busca mantém índices otimizados para consultas aproximadas.

Camada de controle

Controla casos semelhantes ou incompletos antes da ativação. Pode armazenar justificativas, decisões e histórico de alterações.

Divisão visual de responsabilidades



IA e dados

- saneamento, normalização e padronização dos textos;
- autocomplete, fuzzy search, stemming e ranking;
- MongoDB, Atlas Search, índices e migração;
- identificação de duplicidades e cálculo de similaridade;
- matriz de-para e qualidade dos dados.



Java / JavaFX

- aplicação desktop, telas e seletores;
- FXML, CSS, estados visuais e acessibilidade;
- consumo da API, debounce e tratamento de erros;
- validações de interface e feedback ao usuário;
- fluxo de cadastro, busca e reutilização de itens.

A API funciona como ponte entre as duas frentes: recebe as ações da aplicação Java, aplica as regras de negócio e consulta os serviços de dados e busca.

4.2 Onde entra o Spring Boot

O Spring Boot é a tecnologia recomendada para implementar a API de padronização na camada de aplicação. Ele não substitui o JavaFX, o MongoDB ou o Atlas Search: funciona como o backend intermediário entre a aplicação desktop e os serviços de dados.

O fluxo principal fica assim:

```
JavaFX  
| HTTPS / JSON / REST
```

▼
Spring Boot – API de padronização

- Controllers: endpoints REST
- Services: regras de negócio e fluxo de cadastro
- Validation: payload, permissões e limite de 40 caracteres
- Similarity Service: normalização, fuzzy search e decisão
- Audit Service: usuário, data, hora e exceções
- Repositories/Clients: MongoDB, Atlas Search e integrações
-
- MongoDB: fonte oficial dos dados
- Atlas Search: índice e pesquisa textual
- SAP: integração futura

Na prática, os endpoints descritos na seção 10 seriam implementados por controllers Spring, enquanto as regras não devem ficar dentro desses controllers. Uma separação inicial pode ser:

- **Controller**: recebe a requisição e devolve a resposta HTTP;
- **Service**: executa a regra de negócio, como preview, similaridade, idempotência e controle de exceções;
- **Repository**: persiste e consulta os documentos no MongoDB;
- **SearchClient**: encapsula o Atlas Search e permite trocar o mecanismo por OpenSearch ou Elasticsearch no futuro;
- **DTO**: define os contratos de entrada e saída da API;
- **Security**: integra OAuth 2.0/OIDC e aplica as permissões no servidor.

O Spring Boot também pode concentrar tratamento de erros, logs estruturados, configuração por ambiente, testes de integração e documentação dos endpoints. Assim, o JavaFX permanece focado na experiência do usuário, enquanto o backend garante que as mesmas regras sejam aplicadas por qualquer cliente — desktop, integração futura com o SAP ou outra aplicação.

5. Banco de dados: escolha e justificativa

5.1 Por que usar NoSQL

O domínio é composto principalmente por catálogos, descrições, códigos e registros de composição. Não há, no primeiro momento, necessidade de dezenas de tabelas altamente normalizadas nem de consultas relacionais complexas.

Um banco orientado a documentos oferece:

- flexibilidade para evoluir o modelo;
- armazenamento natural de documentos de catálogo;
- facilidade para importar dados heterogêneos;
- boa integração com aplicações Java;
- escalabilidade horizontal;
- integração com busca textual;
- possibilidade de adicionar metadados sem remodelar várias tabelas.

NoSQL não significa ausência de modelo. As coleções devem possuir contrato de dados, validações, nomenclatura e regras de evolução.

5.2 Alternativa recomendada: MongoDB Atlas Search

O MongoDB Atlas é uma opção adequada para o primeiro produto porque reúne armazenamento documental e pesquisa textual em uma mesma plataforma.

Vantagens:

- menor complexidade operacional;
- integração direta entre documentos e índices de busca;
- suporte a busca por texto, autocomplete e fuzzy search;
- boa aderência a APIs Java;
- possibilidade de escalar conforme o uso;
- menos componentes para monitorar;
- facilidade para manter o documento completo junto de seus metadados.

Pontos de atenção:

- verificar custos do serviço gerenciado;
- validar o suporte linguístico necessário para português;
- testar a configuração de analisadores, tokenização e stemming;
- compreender limites e comportamento do fuzzy search;
- manter o banco como fonte oficial e tratar o índice como estrutura derivada.

5.3 Alternativa: Elasticsearch ou OpenSearch

Elasticsearch e OpenSearch são motores especializados em busca e oferecem grande controle sobre análise textual, índices, relevância e observabilidade.

Vantagens:

- alto grau de configuração do analisador;
- recursos maduros de relevância e ranking;
- suporte a agregações e análises mais avançadas;
- boa capacidade para grandes volumes de consultas;
- possibilidade de separar o armazenamento operacional do mecanismo de busca.

Desvantagens:

- maior complexidade de implantação e operação;
- necessidade de sincronizar banco e índice;
- mais componentes para segurança, backup e monitoramento;
- maior esforço de desenvolvimento e troubleshooting;
- risco de inconsistência se a atualização do índice falhar.

5.4 Recomendação

Para o protótipo e a primeira versão, recomenda-se **MongoDB Atlas com Atlas Search**, pois reduz a quantidade de componentes e acelera a entrega.

Elasticsearch ou OpenSearch devem ser considerados quando houver necessidade comprovada de maior escala, relevância altamente customizada, análise avançada ou independência entre armazenamento e busca.

A API deve abstrair o mecanismo de busca. Assim, trocar o Atlas Search por OpenSearch no futuro não exige alterar a interface.

6. Tecnologias de busca textual

6.1 Busca exata

Compara o texto pesquisado com um valor armazenado sem tolerância a variação. É útil para:

- IDs;

- códigos completos;
- valores padronizados como `HR`, `UN` e `M2`;
- consultas em que a precisão é mais importante que a descoberta.

6.2 Prefix search e autocomplete

Retorna resultados a partir do início de uma palavra ou sequência. É o comportamento esperado enquanto o comprador digita.

Exemplo:

```
Entrada: mec
Sugestões: Mecânica Industrial, Mecânico de Manutenção, Manutenção Eletromecânica
```

O autocomplete reduz o tempo de preenchimento e evita que o usuário precise digitar a descrição completa.

6.3 Fuzzy search

Fuzzy search encontra textos próximos mesmo quando existe erro de digitação, troca de caracteres ou pequena diferença ortográfica.

Exemplos conceituais:

- `manutencao` encontra `manutenção`;
- `eletromecanica` encontra `eletromecânica`;
- `mecanico manutencao` encontra variações com palavras invertidas;
- `unid medida` encontra `unidade de medida`.

O fuzzy search deve ser usado com cuidado: uma distância muito permissiva gera falsos positivos; uma distância muito restrita deixa passar duplicidades.

6.4 Stemming

Stemming reduz palavras para uma forma comum, aproximando variações morfológicas. Pode fazer com que palavras como `manutenção`, `manutenções` e formas relacionadas sejam consideradas próximas, dependendo do analisador linguístico.

Stemming não é sinônimo de correção ortográfica. Ele deve ser combinado com normalização, tokenização e fuzzy search.

6.5 Normalização

Antes de indexar e comparar, recomenda-se normalizar o texto:

- converter para minúsculas;
- remover acentos para uma chave de comparação, preservando o texto original para exibição;
- remover espaços duplicados;
- uniformizar separadores;
- tratar abreviações conhecidas;
- padronizar unidades e siglas;
- remover pontuação irrelevante;
- preservar números e códigos relevantes.

Devem ser mantidos dois valores quando necessário:

- `descricaoOriginal` : texto exibido e auditável;
- `descricaoNormalizada` : texto usado para comparação e busca.

6.6 Índices por ID e por texto

Recomenda-se indexar ID e texto de maneira separada ou em campos claramente identificados:

- índice de igualdade/prefixo para ID;
- índice textual para nome ou descrição;
- índice combinado apenas se os testes mostrarem ganho de relevância.

Separar os campos facilita controlar pesos. Uma correspondência exata no ID deve ter prioridade sobre uma semelhança textual baixa.

6.7 Ranking recomendado

O ranking pode combinar:

1. correspondência exata de ID;
2. correspondência de prefixo;
3. correspondência exata de texto normalizado;
4. correspondência textual por tokens;

5. fuzzy score;
6. relevância por uso ou status ativo.

Exemplo de política inicial:

- score `>= 0,90`: provável duplicidade, bloquear ou exigir justificativa;
- score entre `0,75` e `0,89`: apresentar alerta e encaminhar para decisão;
- score abaixo de `0,75`: permitir continuidade, mantendo a validação de negócio.

Esses valores são pontos de partida. Devem ser calibrados com uma amostra real rotulada por especialistas.

7. Modelo de dados

A solução usa quatro coleções de componentes e uma coleção principal de códigos completos.

7.1 Coleção `grupos_servico`

Representa o Grupo de Serviço, ou G.S.

```
{
  "_id": "001",
  "nome": "Eletromecânica",
  "nomeNormalizado": "eletromecanica",
  "origem": "MIGRACAO"
}
```

7.2 Coleção `codigos_fornecedor`

Representa o Código ou classificação de fornecedor, ou C.F.

```
{
  "_id": "001",
  "nome": "Fornecedor de manutenção industrial",
  "nomeNormalizado": "fornecedor manutencao industrial",
  "origem": "MIGRACAO"
}
```

7.3 Coleção **codigos_servico**

Representa o Código de Serviço, ou C.S. Deve conter a descrição submetida à regra de até 40 caracteres.

```
{
  "_id": "000001",
  "descricao": "Manutenção mecânica normal",
  "descricaoNormalizada": "manutencao mecanica normal",
  "quantidadeCaracteres": 29,
  "origem": "MIGRACA0",
  "legacyIds": ["4521", "MEC.MANUT.NORM"]
}
```

7.4 Coleção **unidades_medida**

Representa a Unidade de Medida, ou U.M.

```
{
  "_id": "01",
  "nome": "Hora",
  "sigla": "HR",
  "nomeNormalizado": "hora",
  "origem": "MIGRACA0"
}
```

7.5 Coleção **codigos_completos**

Registra a composição dos quatro componentes.

```
{
  "_id": "00100100000101",
  "idCompleto": "00100100000101",
  "gsId": "001",
  "cfId": "001",
  "csId": "000001",
  "umId": "01",
  "descricaoSnapshot": "Eletromecânica | Fornecedor de manutenção industrial | Manutenção mecânica normal | HR",
}
```

```
"usuarioCriacaoId": "user-origem-ou-ip",  
"createdAt": "2026-10-03T23:24:40-03:00",  
"origem": "APLICACAO",  
"legacyIds": []  
}
```

7.6 Por que guardar os quatro IDs separados

Mesmo existindo um `idCompleto`, os quatro IDs devem ser mantidos em campos próprios porque isso permite:

- buscar todos os serviços de um determinado grupo;
- filtrar por fornecedor ou classificação;
- localizar todos os serviços de uma unidade de medida;
- criar índices eficientes;
- alterar a forma de composição do código no futuro;
- auditar a origem de cada componente;
- evitar depender de parsing de uma string concatenada.

O `idCompleto` é útil para integração e identificação externa, mas não deve ser a única fonte de verdade.

8. Estratégia de IDs

8.1 Requisitos

Os IDs devem ser:

- únicos;
- estáveis após a criação;
- fáceis de referenciar pela API;
- independentes da posição do registro no arquivo de origem;
- preservados em migrações;
- compatíveis com integrações externas.

8.2 IDs dos componentes

Para os componentes sem ID no legado, a aplicação deve gerar IDs automaticamente. Duas opções são adequadas:

- UUID, quando a prioridade é unicidade distribuída;
- sequência controlada, quando a prioridade é legibilidade e compatibilidade operacional.

Recomendação para o projeto: usar IDs técnicos estáveis e manter os IDs legados em um campo separado. O ID exibido ao usuário pode ser uma chave mais amigável, desde que nunca seja reutilizada.

8.3 ID completo

O código completo deve ser composto pelos quatro IDs em posições fixas, sem siglas ou pontuação. Considerando larguras de 3, 3, 6 e 2 caracteres, o exemplo é:

```
00100100000101
```

Como os IDs possuem tamanho fixo, o delimitador não é necessário. Os campos `gsId`, `cfId`, `csId` e `umId` continuam separados no banco para facilitar filtros e auditoria.

O identificador interno não deve ser confundido com a descrição do item no SAP. É necessário distinguir:

- limite da descrição do item no SAP;
- tamanho do identificador interno da aplicação;
- tamanho do código completo de integração.

Se o sistema externo exigir um código curto, deve ser criado um identificador externo separado, mantendo o `idCompleto` interno sem truncamento.

8.4 Unicidade

A combinação dos quatro componentes deve possuir uma restrição única. Antes de inserir, a API deve verificar a existência e o banco deve reforçar a regra com um índice único.

9. Saneamento e migração da base histórica

9.1 Etapas

1. receber os arquivos e identificar o layout;
2. preservar uma cópia imutável da origem;
3. mapear colunas para o novo modelo;
4. validar tipos, campos obrigatórios e tamanho;
5. normalizar os textos;
6. identificar duplicidades exatas;
7. identificar duplicidades aproximadas;
8. agrupar candidatos;
9. definir o registro canônico;
10. gerar IDs para componentes sem ID;
11. gravar os registros consolidados;
12. gravar o relacionamento antigo-novo;
13. validar contagens e amostras;
14. liberar a base para uso da aplicação.

9.2 Cópia de origem

Os dados originais nunca devem ser sobrescritos durante o saneamento. É recomendável manter:

- arquivo de origem;
- data e lote da importação;
- hash do arquivo, quando aplicável;
- versão do algoritmo de normalização;
- usuário ou processo responsável;
- resultado da validação.

9.3 Remoção de duplicatas

A remoção deve acontecer em camadas:

1. **Duplicata exata:** mesma chave ou mesma descrição normalizada.

2. **Duplicata provável:** score de similaridade acima do limite definido.
3. **Duplicata pendente:** casos ambíguos enviados para revisão humana.

Não é recomendável apagar imediatamente os candidatos. O registro consolidado deve apontar para os registros antigos por meio de `legacyIds` ou de uma coleção de relacionamento.

9.4 Escolha da descrição canônica

Quando vários registros representam o mesmo item, a descrição mantida deve ser:

- abrangente, mas não excessivamente longa;
- clara para um comprador;
- compatível com o limite do SAP;
- livre de código interno de fornecedor quando esse código não tiver valor semântico;
- consistente com o vocabulário da categoria;
- revisada quando o algoritmo não conseguir escolher com segurança.

Uma regra automática pode priorizar a descrição mais completa e informativa, mas a decisão final para os casos críticos deve ser humana.

9.5 Dados ausentes

Campos inexistentes na base histórica, como usuário cadastrador e timestamp, devem ser nulos ou receber valores explicitamente identificados, como:

- `usuarioCriacaoId: null`;
- `createdAt: null`;
- `origem: "MIGRACAO"`;
- `legacyImportedAt: data da importação`.

Não se deve preencher dados desconhecidos com um usuário ou data fictícia.

9.6 Matriz de-para

A matriz de-para preserva a ligação entre o legado e o novo padrão.


```
{
  "legacyId": "MEC.MANUT.NORM",
  "legacySource": "ARQUIVO_SAP_2026_01",
  "newEntityType": "CS",
  "newId": "000001",
  "decision": "CONSOLIDADO",
  "reason": "Duplicidade semântica",
  "reviewedBy": "user-123",
  "reviewedAt": "2026-10-03T23:24:40-03:00"
}
```

Essa estrutura permite preservar o histórico de gastos e direcionar referências antigas para o código padronizado.

10. API e regras de negócio

A interface não deve conversar diretamente com o MongoDB. Deve consumir uma API, preferencialmente REST/JSON na primeira versão.

10.1 Endpoints sugeridos

```
GET /api/v1/grupos-servico?query=mec
GET /api/v1/codigos-fornecedor?query=manut
GET /api/v1/codigos-servico?query=mecanica
GET /api/v1/unidades-medida?query=hr
POST /api/v1/codigos-completos/preview
POST /api/v1/codigos-completos
POST /api/v1/itens/similaridade
```

10.2 Preview antes de salvar

Antes de gravar, a API deve retornar uma prévia contendo:

- composição dos quatro IDs;
- descrição final;
- contagem de caracteres;
- existência de código completo igual;
- itens similares encontrados;

- score de similaridade;
- decisão recomendada: `PERMITIR`, `ALERTAR` ou `REVISAR`.

10.3 Idempotência

O endpoint de criação deve ser idempotente para evitar duplicação causada por duplo clique ou repetição de requisição. No cliente, o botão pode ser desabilitado após o clique e a tela pode redirecionar após uma resposta bem-sucedida; no servidor, recomenda-se manter o controle por `idempotency-key` para tratar retentativas.

10.4 Validações obrigatórias

- quatro componentes selecionados;
 - componentes existentes;
 - descrição dentro do limite;
 - composição ainda não cadastrada;
 - usuário autenticado;
 - permissão compatível com a operação;
 - score de similaridade tratado conforme a política;
 - payload válido e seguro.
-

11. Interface gráfica JavaFX

11.1 Por que JavaFX

JavaFX é recomendado para uma aplicação desktop moderna dentro da trilha de Java porque oferece:

- componentes gráficos ricos;
- suporte a CSS para estilização;
- separação entre layout e lógica com FXML;
- propriedades observáveis e binding;
- tabelas, listas, diálogos e controles de entrada;
- integração com tarefas assíncronas;

- melhor capacidade de criar uma experiência consistente que o Swing tradicional.

11.2 JavaFX versus Swing/JFrame

`JFrame` pertence ao Swing, que é uma tecnologia madura e ainda funcional. Porém, para este protótipo, JavaFX tende a ser mais adequado por facilitar:

- autocomplete customizado;
- atualização visual baseada em estado;
- temas claro e escuro;
- validações visuais;
- layouts responsivos;
- manutenção de telas por FXML e CSS.

Swing pode ser escolhido se houver legado corporativo ou bibliotecas internas obrigatórias. A decisão não deve misturar `JFrame` e JavaFX sem necessidade.

11.3 Organização do cliente

Uma estrutura recomendada:

```
src/main/java
├── br.com.gerdau.servicos
│   ├── App.java
│   ├── controller
│   ├── model
│   ├── service
│   ├── client
│   ├── validation
│   └── security
src/main/resources
├── fxml
├── css
└── images
```

11.4 Padrão arquitetural

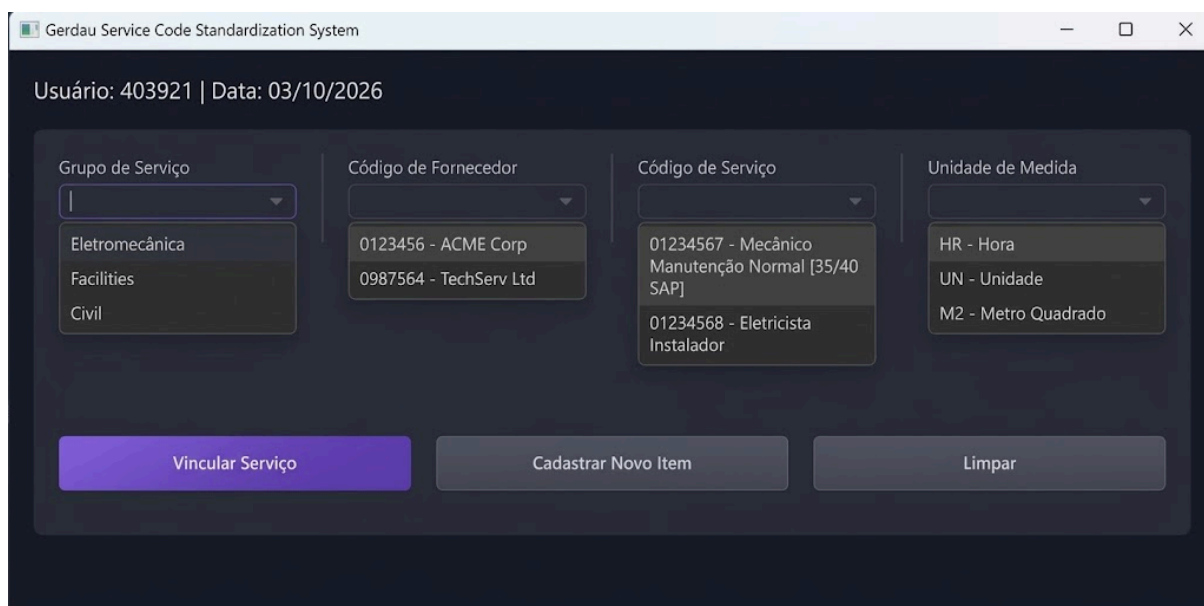
O padrão MVC ou MVVM deve separar:

- **View:** FXML e CSS;
- **Controller/ViewModel:** estado da tela e eventos;

- **Service:** regras de interação com a API;
- **Model/DTO:** representação dos dados;
- **Client HTTP:** comunicação REST;
- **Validation:** validações de campo e mensagens.

A regra de similaridade não deve ficar somente no controller. A interface pode antecipar feedback, mas a API deve repetir a validação.

11.5 Tela de seleção de serviço completo



A tela deve apresentar quatro componentes lado a lado:

1. Grupo de Serviço;
2. Código do Fornecedor;
3. Código do Serviço;
4. Unidade de Medida.

Cada componente deve ter:

- campo de pesquisa;
- lista de sugestões;
- indicação do ID;
- descrição legível;

- estado de carregamento;
- mensagem para nenhum resultado;
- possibilidade de limpar a seleção.

Após a seleção dos quatro itens, a tela deve mostrar a prévia do código completo e o botão de salvar.

11.6 Busca enquanto o usuário digita

A cada caractere, a aplicação pode consultar a API, mas deve aplicar:

- atraso curto por debounce, por exemplo 250–400 ms;
- cancelamento da requisição anterior;
- mínimo de caracteres, quando apropriado;
- limite de resultados;
- indicação visual de carregamento;
- tratamento de indisponibilidade da API.

Sem debounce, a interface pode gerar excesso de chamadas e uma experiência instável.

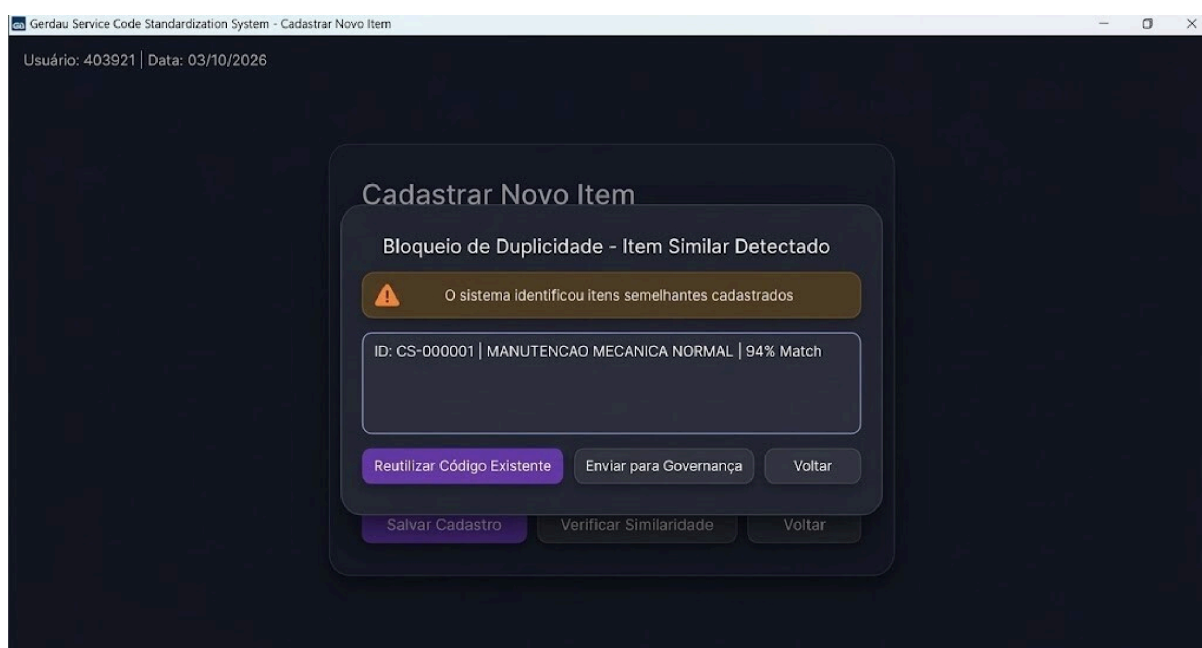
11.7 Tela de cadastro de item

A tela deve permitir selecionar o tipo:

- Grupo de Serviço;
- Código de Fornecedor;
- Código de Serviço;
- Unidade de Medida.

Depois, deve permitir informar nome, sigla ou descrição conforme o tipo. Ao sair do campo ou clicar em validar, a aplicação consulta a API e apresenta itens similares.

11.8 Tela de similaridade



Cada resultado deve mostrar:

- descrição atual;
- ID;
- score de similaridade;
- motivo do alerta, quando aplicável;
- ação de reutilizar;
- ação de enviar para governança;
- ação de registrar uma exceção.

A tela não deve exibir apenas um percentual sem contexto. O usuário precisa entender por que o item foi considerado semelhante.

Caso necessário o envio a governança, uma validação humana por parte da gerdau seria necessária para manter a ordem, além de ser obrigatório uma descrição do porquê criar aquele item e não utilizar outro já existente.

A interface é exibida em um fundo escuro com uma borda decorativa em tons de roxo. No topo, há uma caixa de alerta amarela com um ícone de 'i' e o texto: **Atenção:** Este cadastro passará por validação humana pela Equipe de Governança de Dados. Abaixo, o título **Envio para Governança** está em branco. O formulário contém os seguintes campos: 'Classe:' (vazio), 'Código do Serviço (C.S)' (vazio), 'Descrição:' (contendo o texto 'MECANICO MANUTENCAO COM ESPECIALIZACAO'), e 'Justificativa da Solicitação:' (contendo o texto 'Serviço não atende às especificações dos itens existentes' em uma caixa de texto com barra de rolagem). Na base, há dois botões: 'Voltar' (transparente) e 'Confirmar Envio para Governança' (verde sólido).

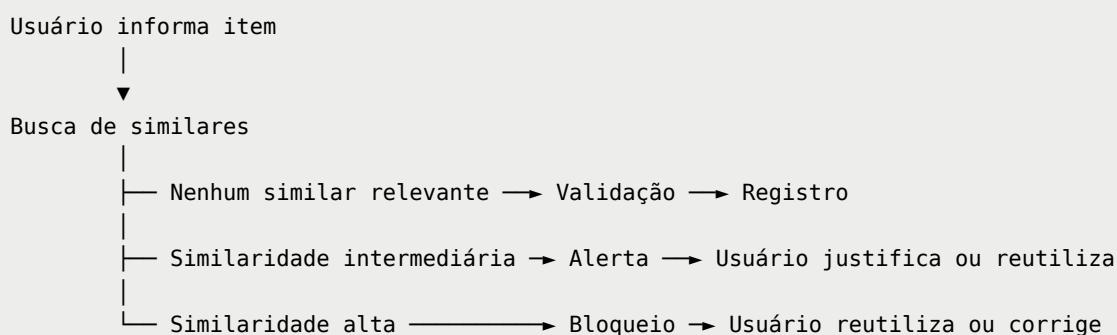
11.9 Acessibilidade e usabilidade

- permitir navegação por teclado;
- manter foco visível;
- usar mensagens de erro objetivas;
- diferenciar alerta, erro e bloqueio;
- não depender apenas de cor;

- apresentar unidades e siglas de forma clara;
- evitar telas com excesso de informações;
- informar quando o resultado é aproximado;
- preservar os dados digitados em caso de erro de rede.

12. Controle e permissões

12.1 Fluxo recomendado



12.2 Perfis sugeridos

- **Consultor:** pesquisa e visualiza itens.
- **Solicitante:** cria itens ou solicitações de novo item.
- **Administrador de dados:** consolida duplicidades e corrige cadastros.
- **Administrador técnico:** configura índices, parâmetros e integrações.

12.3 Registro de exceções

Quando o usuário insistir em um item sinalizado, o sistema deve registrar:

- solicitante;
- data e hora;
- payload original;
- similares encontrados;
- justificativa;
- decisão tomada;

- observação da decisão.

Não haverá uma etapa separada de aprovador; o registro serve para rastreabilidade e revisão posterior.

12.4 Bloqueio versus alerta

Não é recomendável bloquear todos os casos desde o primeiro dia. A implantação pode começar em modo de observação:

1. registrar os candidatos;
 2. medir falsos positivos e falsos negativos;
 3. ajustar score e regras;
 4. ativar alertas;
 5. ativar bloqueios apenas onde houver confiança.
-

13. Segurança

13.1 Autenticação

A API deve autenticar o usuário. Como a aplicação é desktop, a estratégia pode usar:

- provedor corporativo com OAuth 2.0/OpenID Connect;
- token de curta duração;
- renovação segura;
- logout e limpeza de credenciais locais.

Não se deve confiar apenas no ID ou IP informado pelo cliente. O servidor deve obter a identidade do token autenticado e registrar o IP apenas como metadado complementar.

13.2 Autorização

Cada operação deve verificar o perfil do usuário no servidor. Ocultar um botão na interface não é controle de segurança.

13.3 Comunicação

- usar HTTPS;

- validar certificados;
- evitar segredos no código;
- não registrar tokens em logs;
- proteger variáveis de ambiente;
- limitar origem e rede de acesso à API;
- aplicar rate limiting nos endpoints de busca e cadastro.

13.4 Dados pessoais

O usuário cadastrador e o IP podem ser dados pessoais ou identificáveis. O projeto deve:

- restringir acesso aos logs;
 - registrar somente o necessário;
 - definir retenção;
 - evitar exibição pública do IP;
 - documentar finalidade e responsabilidade;
 - alinhar o tratamento à política de privacidade e à LGPD.
-

14. Performance e escalabilidade

14.1 Busca

- limitar o tamanho da resposta;
- paginar resultados quando necessário;
- aplicar debounce no cliente;
- manter índices apropriados;
- evitar trazer documentos completos para autocomplete;
- retornar somente `id`, nome, descrição e score na busca.

14.2 Escrita

- usar validação de esquema;
- garantir índice único;

- usar operações atômicas quando houver criação de componente e código completo;
- tornar o endpoint idempotente;
- evitar duplicação em retentativas.

14.3 Índices sugeridos

Na coleção de componentes:

- índice por `_id`;
- índice textual por nome ou descrição normalizada;
- índice de origem, se houver relatórios de migração.

Na coleção de códigos completos:

- índice único em `idCompleto`;
- índice em `gsId`;
- índice em `cfId`;
- índice em `csId`;
- índice em `umId`;
- índice composto conforme os filtros mais frequentes;
- índice em `createdAt` somente se houver consultas temporais relevantes.

Índices devem ser validados por métricas e consultas reais. Índice demais aumenta custo de escrita e armazenamento.

15. Observabilidade e operação — complementar



Seção complementar: estes controles são importantes em serviços, mas não são essenciais para a primeira implementação.

Esses controles são importantes em serviços, mas podem ser implementados gradualmente. Para a primeira versão, recomenda-se começar pelos itens mais simples, como logs estruturados, tempo de resposta, erros da API e falhas de integração.

A solução deve possuir visibilidade sobre:

- tempo de resposta das buscas;
- taxa de erro da API;
- quantidade de cadastros bloqueados;
- quantidade de casos revisados;
- score médio das similaridades;
- tempo de decisão;
- erros de sincronização do índice;
- falhas de integração com o SAP;
- volume de registros importados;
- duplicidades consolidadas.

15.1 Logs estruturados

Os logs devem ser estruturados em JSON ou formato equivalente e conter correlação por requisição. Não devem conter senha, token, descrição sensível desnecessária ou payload completo quando isso não for necessário.

15.2 Métricas de sucesso

- redução de novos cadastros duplicados;
- percentual de consultas resolvidas sem novo cadastro;
- tempo médio para localizar um serviço;
- percentual de itens concluídos sem retrabalho;
- precisão dos alertas de similaridade;
- cobertura da matriz de-para;
- disponibilidade da API.

16. Testes — complementar



Seção complementar: estes testes ajudam a evoluir e validar a solução, mas podem ser implementados gradualmente após o protótipo inicial.

16.1 Testes unitários

Validar isoladamente:

- normalização de texto;
- contagem de caracteres;
- montagem do ID completo;
- regras de permissão;
- classificação por score;
- tratamento de campos nulos;
- conversão de dados legados.

16.2 Testes de integração

Verificar:

- comunicação JavaFX com a API;
- API com o banco;
- API com o mecanismo de busca;
- autenticação;
- criação idempotente;
- atualização do índice;
- fluxo de exceções.

16.3 Testes de similaridade

Criar um conjunto rotulado com:

- duplicatas verdadeiras;
- itens parecidos, mas diferentes;
- erros de digitação;
- acentos ausentes;
- abreviações;
- palavras em ordem diferente;
- descrições muito curtas;

- códigos com números.

Medir precisão, revocação e taxa de falsos bloqueios antes de definir o limiar definitivo.

16.4 Testes de interface

- seleção de cada componente;
- pesquisa sem resultados;
- indisponibilidade da API;
- duplo clique em salvar;
- recuperação após timeout;
- teclado e foco;
- limite de 40 caracteres;
- apresentação de resultados similares.

16.5 Testes de migração

- contagem de registros antes e depois;
- quantidade de duplicatas encontradas;
- amostra manual do de-para;
- existência de todos os componentes referenciados;
- ausência de códigos completos inválidos;
- preservação dos dados históricos.

17. Implantação e ambientes

Recomenda-se separar pelo menos:

- **desenvolvimento:** dados fictícios e testes locais;
- **homologação:** cópia controlada e anonimizada para validação;
- **produção:** dados oficiais e permissões restritas.

Configurações como URL da API, credenciais, thresholds e nomes de índices devem ficar fora do código-fonte, usando variáveis de ambiente ou um serviço corporativo de configuração.

O pipeline de entrega deve executar:

1. compilação;
2. testes unitários;
3. testes de integração;
4. análise de qualidade;
5. empacotamento da aplicação JavaFX;
6. publicação da API;
7. migrações ou atualizações de índice controladas;
8. validação pós-deploy.

Backups devem incluir o banco operacional, configurações de índices, arquivos de origem e matriz de-para.

18. Roadmap de implementação

Fase 1 — Descoberta e preparação

- confirmar layout dos dados;
- levantar regras atuais do SAP;
- definir glossário das categorias;
- separar amostra rotulada;
- escolher MongoDB Atlas como opção inicial;
- definir contratos da API.

Fase 2 — Modelo e migração inicial

- criar as coleções;
- gerar IDs técnicos;
- implementar normalização;
- importar dados históricos;
- gerar matriz de-para;
- validar duplicidades com especialistas.

Fase 3 — Busca

- configurar índices por ID e texto;
- testar autocomplete;
- testar fuzzy search;
- testar stemming e analisadores em português;
- calibrar score e pesos;
- expor endpoints de consulta.

Fase 4 — Aplicação JavaFX

- construir tela de seleção completa;
- construir cadastro de item;
- integrar autenticação;
- adicionar validações;
- adicionar feedback de similaridade;
- implementar tratamento de erros e timeout.

Fase 5 — Piloto

- iniciar com Eletromecânica, Facilities e Civil;
- acompanhar falsos positivos;
- coletar feedback dos compradores;
- ajustar vocabulário e limites;
- medir redução de duplicidades.

Fase 6 — Expansão

- ampliar categorias;
 - integrar com o SAP;
 - automatizar relatórios;
 - avaliar OpenSearch ou Elasticsearch se necessário;
 - revisar a arquitetura com base no volume real.
-

19. Decisões técnicas resumidas

Tema	Decisão recomendada	Motivo
Banco	MongoDB Atlas	Menor complexidade e boa integração com busca
Busca	Atlas Search	Fuzzy, autocomplete e análise textual próximos dos documentos
Alternativa de busca	OpenSearch/Elasticsearch	Considerar quando houver necessidade de controle avançado e escala
Interface	JavaFX	Melhor separação visual, CSS, FXML e componentes modernos
Comunicação	API REST/JSON	Desacopla a interface do banco e facilita futuras integrações
Modelo	Quatro coleções de componentes + códigos completos	Reuso, filtros e controle por entidade
Similaridade	Normalização + autocomplete + fuzzy + stemming	Combina velocidade, tolerância a erros e variações linguísticas
Identidade	IDs estáveis e matriz de-para	Preserva referências históricas
Cadastro	Preview, validação e idempotência	Evita duplicidade e duplo envio
Controle	Alertas, bloqueios e registro de exceções	Equilibra autonomia e controle
Legado	Importação sem apagar origem	Permite auditoria e reversão
Segurança	OAuth/OIDC, HTTPS e RBAC	Protege identidade, acesso e operações

20. Riscos e mitigações

Risco	Impacto	Mitigação
Fuzzy permissivo demais	Bloqueios indevidos	Calibrar com base rotulada e usar faixas de decisão
Fuzzy restritivo demais	Duplicidades passam	Combinar normalização, stemming, sinônimos e revisão periódica
Índice desatualizado	Busca inconsistente	Monitorar sincronização e permitir reindexação

Risco	Impacto	Mitigação
Descrição acima de 40 caracteres	Falha no SAP	Validar no cliente e no servidor
ID completo excessivamente longo	Incompatibilidade externa	Separar identificador interno de código externo
Usuário não confiável no payload	Auditoria incorreta	Derivar identidade do token no servidor
Banco acessado diretamente pelo desktop	Risco de segurança	Exigir API intermediária
Migração incorreta	Perda de histórico	Preservar origem, lotes, de-para e validações
Excesso de índices	Custo e lentidão de escrita	Criar índices a partir de consultas reais
Dependência excessiva de um fornecedor	Migração difícil	Encapsular busca atrás de uma interface de serviço

21. Entregáveis do projeto

21.1 Relatório técnico

Deve conter:

- diagnóstico do problema;
- objetivos e escopo;
- arquitetura;
- modelo de dados;
- estratégia de saneamento;
- escolha de tecnologias;
- algoritmo de busca;
- controle e permissões;
- segurança;
- testes;
- roadmap;
- riscos e métricas.

21.2 Mapa mental

Deve mostrar apenas os conceitos-chave:

- problema;
- dados;
- saneamento;
- NoSQL;
- fuzzy search;
- stemming;
- JavaFX;
- API;
- controle;
- SAP;
- benefícios.

21.3 Desenho de arquitetura

Deve representar:

- aplicação JavaFX;
- API;
- banco NoSQL;
- índice de busca;
- controle de exceções;
- integração futura com SAP;
- direção das requisições e respostas.

21.4 Protótipo visual

Deve conter:

- tela principal com os quatro seletores;
- autocomplete;
- prévia do código completo;

- tela de cadastro de item;
- alerta de similaridade;
- estado bloqueado;
- mensagem de bloqueio e alerta.

21.5 Materiais visuais

Os artefatos visuais associados ao projeto podem ser utilizados como apoio à apresentação:

- fluxograma operacional;
 - protótipo da tela principal;
 - protótipo da tela de cadastro;
 - mapa mental;
 - vídeo explicativo.
-

22. Conclusão

A solução proposta não é apenas uma tela de cadastro ou uma troca de banco. Ela cria uma camada de qualidade e controle entre o usuário e o catálogo de serviços.

O MongoDB Atlas com busca textual atende bem à primeira versão por reduzir a complexidade operacional. O JavaFX mantém o projeto alinhado à trilha de Java e permite construir uma aplicação desktop com boa experiência de uso. A API desacopla os módulos, centraliza as regras e mantém aberta a possibilidade de substituir o mecanismo de busca no futuro.

A combinação de normalização, fuzzy search, stemming, índices por ID e descrição, validação do limite do SAP, matriz de-para e controle de exceções trata tanto o problema histórico quanto a prevenção de novos erros.

O resultado esperado é um catálogo mais confiável, pesquisável e rastreável, capaz de apoiar a consolidação de compras, a comparação de fornecedores, a análise de gastos e a evolução da qualidade dos dados na Gerdau.